

Sarika Palle

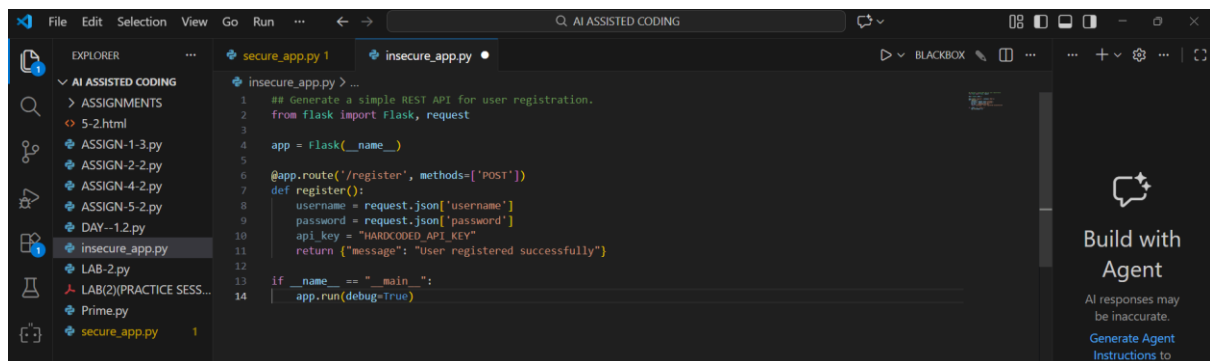
2403a51l33

B-52

Lab 5: Ethical Foundations – Responsible AI Coding Practices

Task Description – 1: Secure API Usage

Prompt: Generate a simple REST API for user registration.

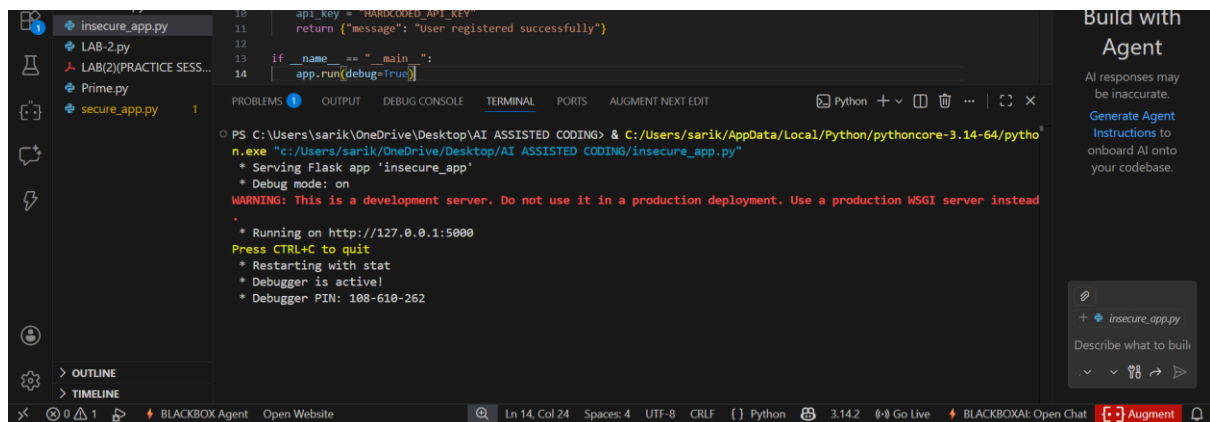


The screenshot shows the Visual Studio Code editor with a file explorer on the left containing various Python files. The main editor window displays a file named `insecure_app.py` with the following Python code:

```
1  ## Generate a simple REST API for user registration.
2  from flask import Flask, request
3
4  app = Flask(__name__)
5
6  @app.route('/register', methods=['POST'])
7  def register():
8      username = request.json['username']
9      password = request.json['password']
10     api_key = "HARDCODED_API_KEY"
11     return ("message": "User registered successfully")
12
13 if __name__ == "__main__":
14     app.run(debug=True)
```

On the right side of the editor, there is a sidebar titled "Build with Agent" with the text "AI responses may be inaccurate." and a button labeled "Generate Agent".

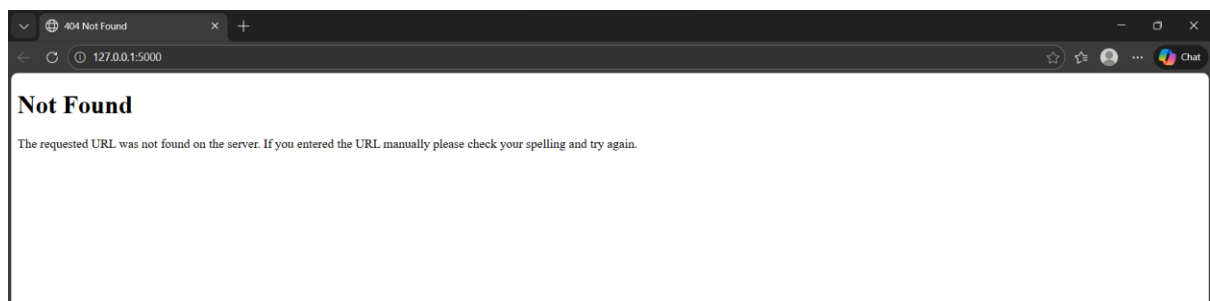
OUTPUT:



The screenshot shows the Visual Studio Code editor with the same file explorer and editor window as before. The main editor window now displays the output of the application in the terminal, showing the following text:

```
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> & C:\Users\sarik\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING\insecure_app.py"
* Serving Flask app 'insecure_app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 108-610-262
```

On the right side of the editor, there is a sidebar titled "Build with Agent" with the text "AI responses may be inaccurate." and a button labeled "Generate Agent".



```

9      password = request.json['password']
10     api_key = "HARDCODED_API_KEY"
11     return {"message": "User registered successfully"}
12
13 if __name__ == "__main__":
14     app.run(debug=True)

```

```

PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/sarik/AppData/Local/Python/pythoncore-3.14-64/python
n.exe "c:/Users/sarik/OneDrive/Desktop/AI ASSISTED CODING/insecure_app.py"

* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 188-610-262
127.0.0.1 - - [20/Jan/2026 21:46:17] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [20/Jan/2026 21:46:17] "GET /favicon.ico HTTP/1.1" 404 -

```

Explanation: You got 404 error because your Flask app does not have a home (/) route, so the browser cannot find that page.

Identified Security Flaws:

1. API key is **hardcoded**, exposing sensitive credentials
2. No authentication or authorization mechanism
3. No input validation (password strength, missing fields)
4. Password stored/used in **plain text**
5. No token-based access control

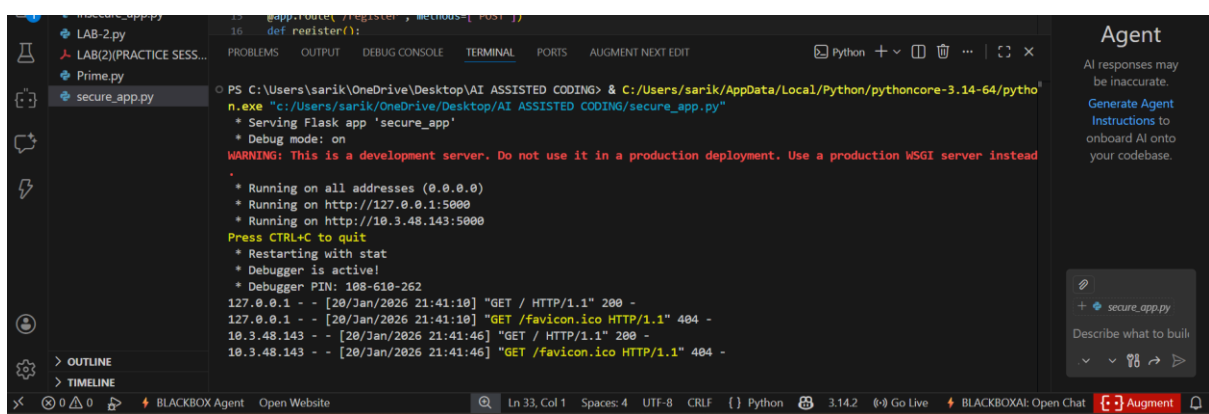
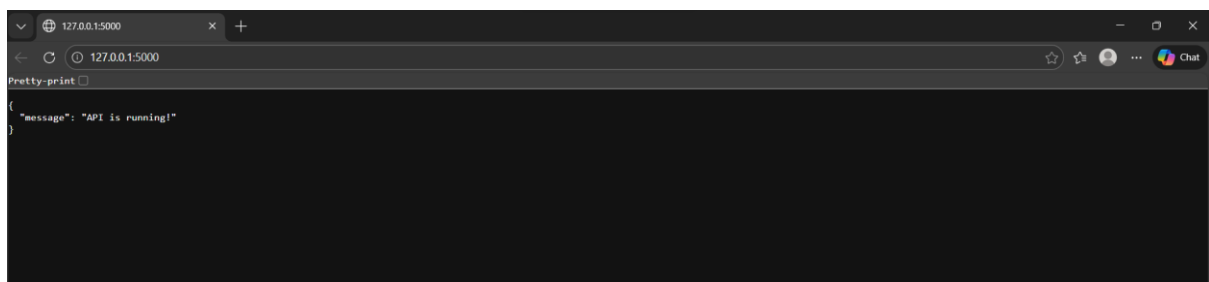
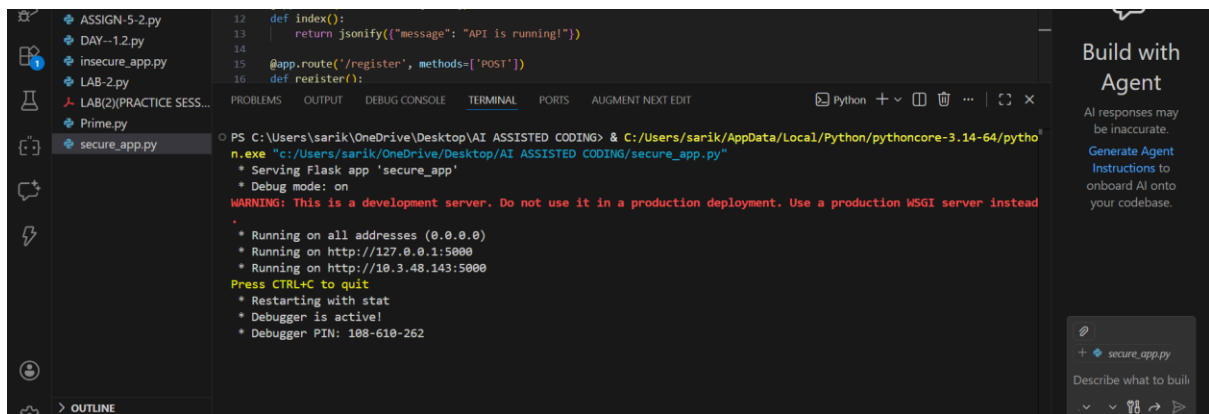
Corrected Secure Version (Token-Based Authentication):

```

1  # Secure API (Corrected - Token-Based Authentication)
2  > from flask import Flask, request, jsonify
3
4  app = Flask(__name__)
5  app.config['SECRET_KEY'] = os.getenv("SECRET_KEY", "mysecretkey")
6
7  @app.route('/', methods=['GET'])
8  def index():
9      return jsonify({"message": "API is running!"})
10
11 @app.route('/register', methods=['POST'])
12 def register():
13     data = request.get_json()
14     if not data or not data.get('username') or not data.get('password'):
15         return jsonify({"error": "Invalid input"}), 400
16     hashed_password = generate_password_hash(data['password'])
17     token = jwt.encode(
18         {
19             'user': data['username'],
20             'exp': datetime.datetime.utcnow() + datetime.timedelta(hours=1)
21         },
22         app.config['SECRET_KEY'],
23         algorithm="HS256"
24     )
25     return jsonify({"token": token})
26
27 if __name__ == "__main__":
28     app.run(debug=True, host="0.0.0.0", port=5000)

```

OUTPUT:

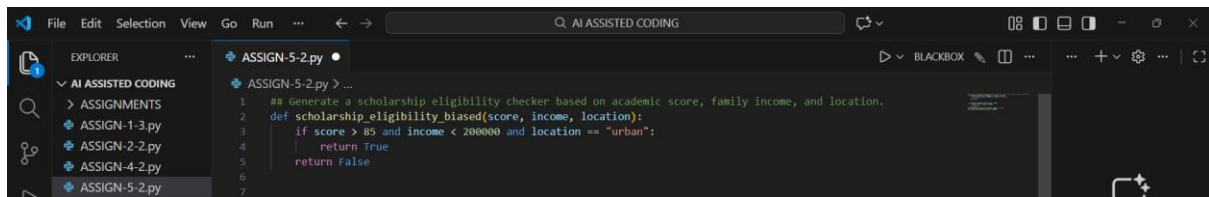


Observations: The initial API code is insecure because it uses a hardcoded API key and does not protect user data. The corrected version improves security by validating inputs, hashing passwords, and using token-based authentication for safer access control.

Task Description – 2: Fair Decision Logic

Prompt: Generate a scholarship eligibility checker based on academic score, family income, and location.

AI-Generated Code:

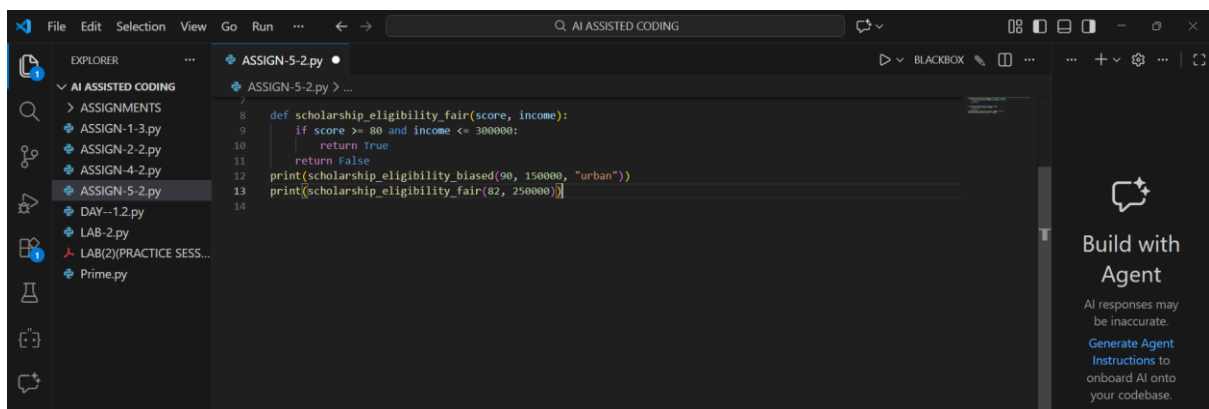


```
1  ## Generate a scholarship eligibility checker based on academic score, family income, and location.
2  def scholarship_eligibility_biased(score, income, location):
3      if score > 85 and income < 200000 and location == "urban":
4          return True
5      return False
6
7
```

Observations:

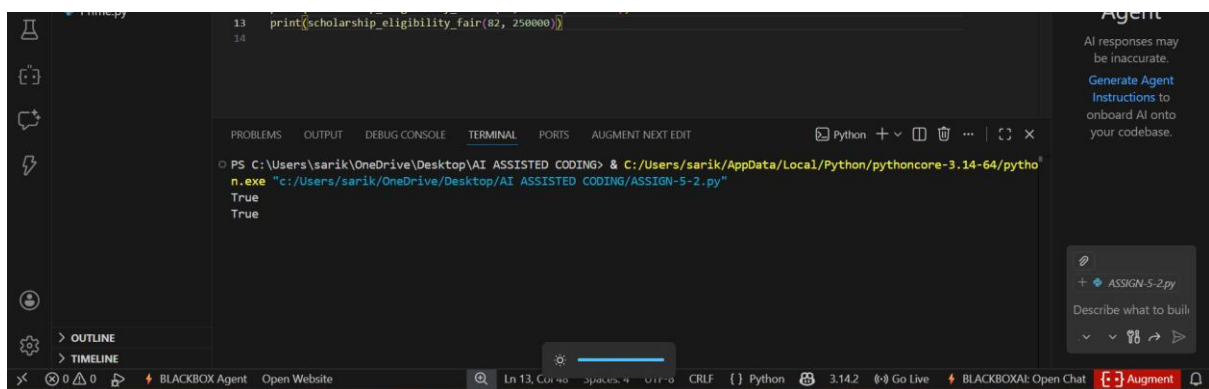
1. The logic unfairly favors urban students
2. Rural or semi-urban students are excluded
3. No flexibility or weighted scoring approach

Improved Version:



```
8  def scholarship_eligibility_fair(score, income):
9      if score >= 80 and income <= 300000:
10         return True
11     return False
12
13     print(scholarship_eligibility_biased(90, 150000, "urban"))
14     print(scholarship_eligibility_fair(82, 250000))
15
```

OUTPUT:

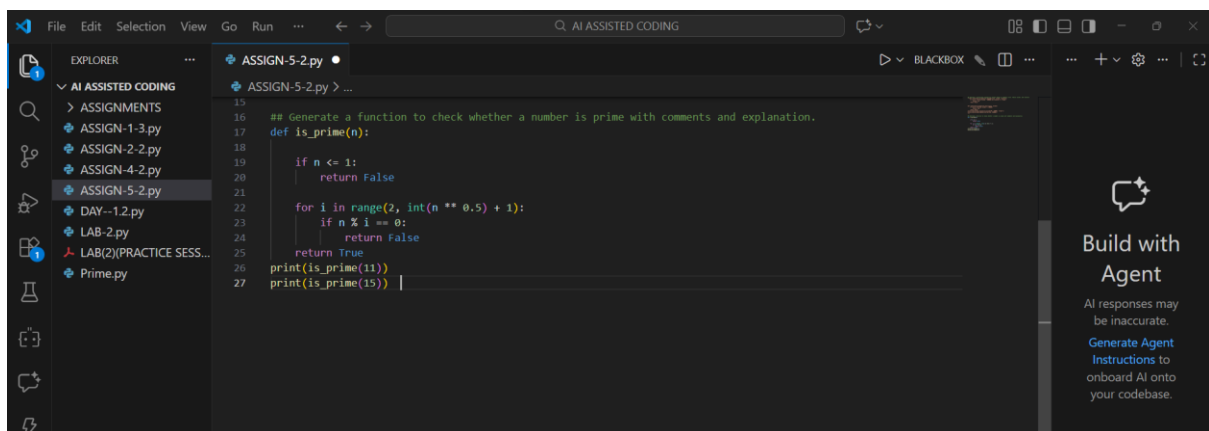


```
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> & C:\Users\sarik\AppData\Local\Python\pythoncore-3.14-64\python
n.exe "c:/Users/sarik/OneDrive/Desktop/AI ASSISTED CODING/ASSIGN-5-2.py"
True
True
```

Explanation: The original logic introduces geographic bias by favoring urban students. Location should not be a deciding factor unless justified by policy. A fair system focuses on merit and economic need. Weighted or threshold-based criteria help ensure equitable access.

Task Description – 3: Explainability

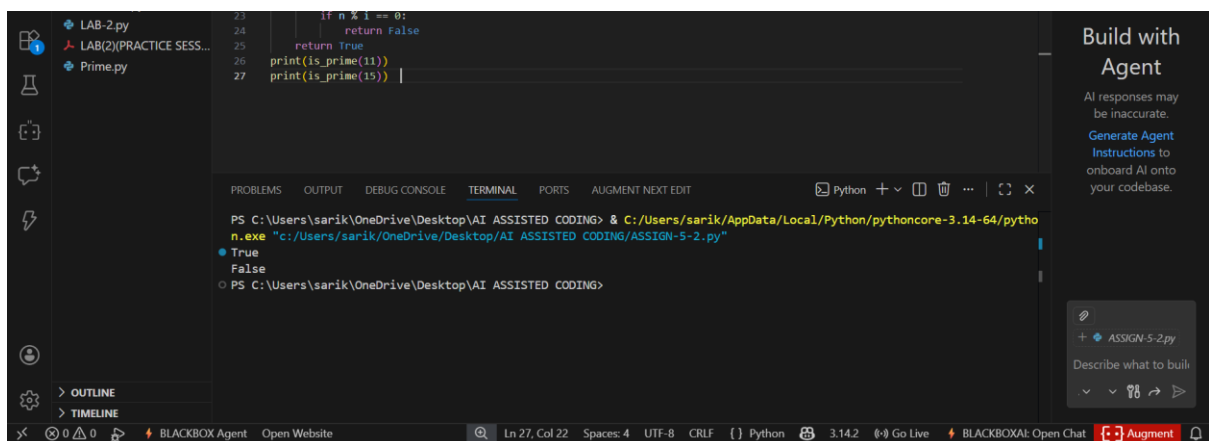
Prompt: Generate a function to check whether a number is prime with comments and explanation.



The screenshot shows the Visual Studio Code editor with a file named 'ASSIGN-5-2.py' open. The code defines a function 'is_prime(n)' that checks if a number is prime. It includes a comment: '## Generate a function to check whether a number is prime with comments and explanation.' The function returns False for n <= 1 and for numbers divisible by any integer from 2 to the square root of n. It then prints the results for 11 and 15. On the right side, there is a sidebar with the text 'Build with Agent' and a note that 'AI responses may be inaccurate.' Below this, there are links to 'Generate Agent' and 'Instructions to onboard AI onto your codebase.'

```
15
16 ## Generate a function to check whether a number is prime with comments and explanation.
17 def is_prime(n):
18
19     if n <= 1:
20         return False
21
22     for i in range(2, int(n ** 0.5) + 1):
23         if n % i == 0:
24             return False
25     return True
26 print(is_prime(11))
27 print(is_prime(15)) ]
```

OUTPUT:



The screenshot shows the Visual Studio Code editor with the same file 'ASSIGN-5-2.py' open. The output of the code is displayed in the terminal window at the bottom. The output shows 'True' for 11 and 'False' for 15. On the right side, there is a sidebar with the text 'Build with Agent' and a note that 'AI responses may be inaccurate.' Below this, there are links to 'Generate Agent' and 'Instructions to onboard AI onto your codebase.'

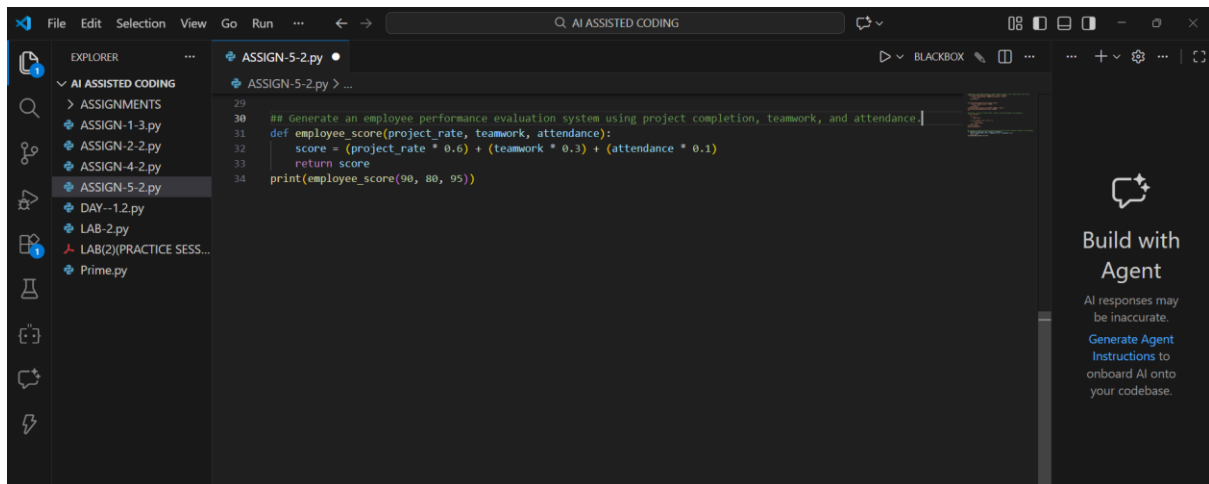
```
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/sarik/AppData/Local/Python/pythoncore-3.14-64/python
n.exe "c:/Users/sarik/OneDrive/Desktop/AI ASSISTED CODING/ASSIGN-5-2.py"
• True
False
○ PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING>
```

Explanation: The function first checks if the number is greater than 1. It then tests divisibility from 2 up to the square root of the number to reduce computation. If any divisor is found, the number is not prime; otherwise, it is prime.

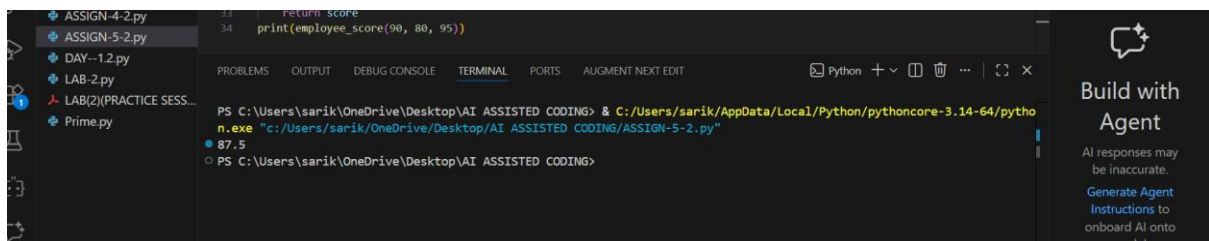
The explanation is clear, correct, and efficient. Inline comments improve readability and help beginners understand the logic easily.

Task Description – 4: Ethical Scoring System

Prompt: Generate an employee performance evaluation system using project completion, teamwork, and attendance.



OUTPUT:



Observations:

1. Heavy weight on project completion may disadvantage collaborative roles
2. Attendance weighting may penalize employees with health or caregiving needs
3. Teamwork score depends on subjective evaluation

The criteria are reasonable but require transparency and flexibility. Ethical systems should allow contextual review and avoid over-reliance on single metrics.

Task Description – 5: Accessibility and Inclusiveness

Prompt: Generate a user feedback form application.

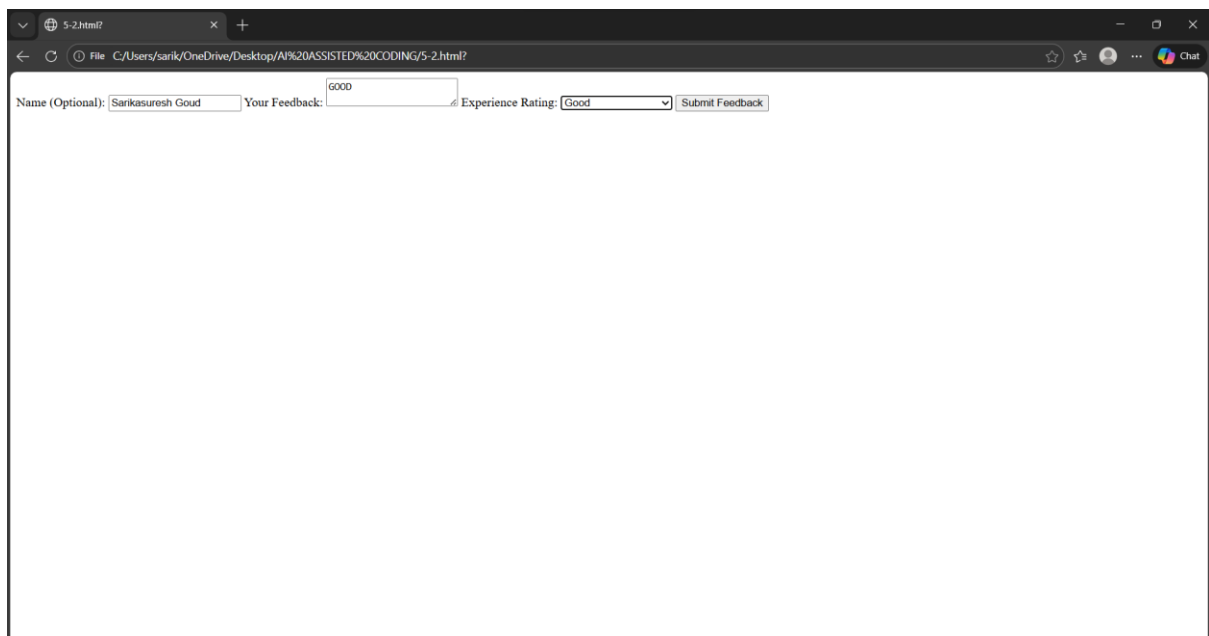
```
<form aria-label="User Feedback Form">
  <label for="name">Name (Optional):</label>
  <input type="text" id="name" aria-required="false">

  <label for="feedback">Your Feedback:</label>
  <textarea id="feedback" aria-required="true"></textarea>

  <label for="rating">Experience Rating:</label>
  <select id="rating">
    <option>Very Good</option>
    <option>Good</option>
    <option>Neutral</option>
    <option>Needs Improvement</option>
  </select>

  <button type="submit">Submit Feedback</button>
</form>
```

OUTPUT:



The screenshot shows a web browser window with the URL `C:/Users/sarik/OneDrive/Desktop/AI%20ASSISTED%20CODING/5-2.html?``. The rendered form is titled "User Feedback Form" and contains the following elements:

- A text input field labeled "Name (Optional):" with the value "Sarikasuresh Goud".
- A text area labeled "Your Feedback:" with the value "GOOD".
- A dropdown menu labeled "Experience Rating:" with the value "Good".
- A "Submit Feedback" button.

Observations: The feedback form uses neutral and inclusive language to avoid exclusion of any user group. Accessibility is enhanced through ARIA labels, optional fields, and simple input options for diverse users.

